

Thoughts on code generation:

1) For some aspects of code generation, the Compute Size pass could have been differently. For example, it would be convenient if `cParamListNode` had a size so that after the return you know how to clean up the stack. Feel free to alter your Compute Size pass as appropriate to make code generation easier. It would be better to do that work in the Compute Size pass instead of allowing that to leak into your code generation pass. I will not specifically look for this separation, so feel free to do it either way.

2) There are several things which are syntactically legal, but it would be difficult to generate correct code. These can be considered undetected semantic errors, and you can avoid them in your testing

a) Functions with no return statement:

```
int foo()  
{  
    int a;  
    a = 5;  
}
```

Since there is no return statement in the source code, no return gets generated in the assembly, and thus the function never returns. Since there are no “void” functions in lang, all functions are semantically required to have a return statement. If it is missing, that is an undetected semantic error.

b) Function calls as statements:

```
int foo() { return 5; }  
foo();
```

The problem here is that the call to `foo()` leaves the return value on the stack. If the call is on the right hand side of an assignment, the assignment removes the return value from the stack, but when the call is by itself, there is no convenient way to know to remove it. In my test code, I will not test this case. We will talk in class about ways to address this issue.